

NEXT-GEN READ ALIGNMENT AND CONTIG QUERY IDENTIFICATION

Research Methods in Biomedical Informatics

Christian Anderson

04 May 2026

CONTENTS

1. The Problem
2. Background
3. My Approach
4. Results
5. Conclusions

THE PROBLEM

Problem

“Create a program that takes as input the set of all next-generation sequencing reads identified in a sample and an initial query sequence and returns the largest sequence contig that can be constructed from the reads that contains the initial query sequence.”

Imagine you want to confirm that a mutation is present in a mouse model of brittle bone disease. You have millions of short DNA fragments from a sequencer and one short sequence you know should be there. How do you find the full surrounding region?

- Researchers validating transgenic knock-in models need to confirm the sequence around a mutation
- There is no reference to compare against
- A targeted assembler anchored on a known query sequence can recover the region of interest directly

BACKGROUND

Genomes can be billions of base pairs long, but sequencing machines can only read hundreds to thousands at a time.

The genome must first be shattered into millions of small fragments, each sequenced independently, then computationally reassembled.

Short reads (Illumina)

150 bp · very accurate

Struggles with repetitive regions

Long reads (PacBio, Oxford Nanopore)

10,000+ bp · less accurate

Spans repetitive regions



Query sequence: **ZZZZZZZZ**

Sequencer reads:

ZZZWZYZY	YYYYYVYYY	XXXXZZZZ
ZZZZZZZZ	XXXXXXXXXX	ZZZZYYYYY
BBBBZBBYB	BBBBZBBYB	DDDDDDDDDD

Output contig: **XXXXXXXXXXZZZZZZZZZZ**YYYYYVYYY

“In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.”

–*The Hobbit* [1]

“In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.”

–*The Hobbit* [1]

Remove non-alphanumeric characters

inaholeinthegroundtherelivedahobbitnotanastydirty...

“In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.”

–*The Hobbit* [1]

Remove non-alphanumeric characters

inaholeinthegroundtherelivedahobbitnotanastydirty...

Simulate Reads by taking random substrings of different length

inahole
intheground
edahobbitno
itnotanastydirty ...

“In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.”

–*The Hobbit* [1]

Remove non-alphanumeric characters

inaholeinthegroundtherelivedahobbitnotanastydirty...

Simulate Reads by taking random substrings of different length

inahole
intheground
edahobbitno
itnotanastydirty ...

Chop into k length 'mers using a sliding window

$k = 5$

inaho, nahol, nahole, aholei, holein, ... itnot, tnota, notan, otana ...

“In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort.”

–*The Hobbit* [1]

Remove non-alphanumeric characters

inaholeinthegroundtherelivedahobbitnotanastydirty...

Simulate Reads by taking random substrings of different length

inahole
intheground
edahobbitno
itnotanastydirty ...

How do we work backwards?

Chop into k length 'mers using a sliding window

$k = 5$

inaho, nahol, nahole, aholei, holein, ... itnot, tnota, notan, otana ...

What is a graph?

A graph is a structure used to encode connections between things.

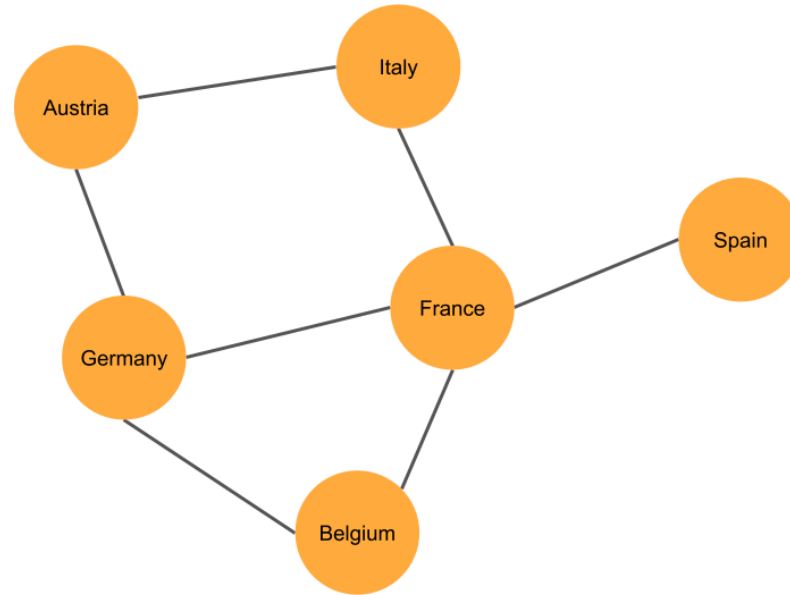


Figure 1: A simple graph used to represent how countries share borders.

Using the *Hobbit* example, I can plot the graph connecting each kmer to every other kmer that has an overlap.

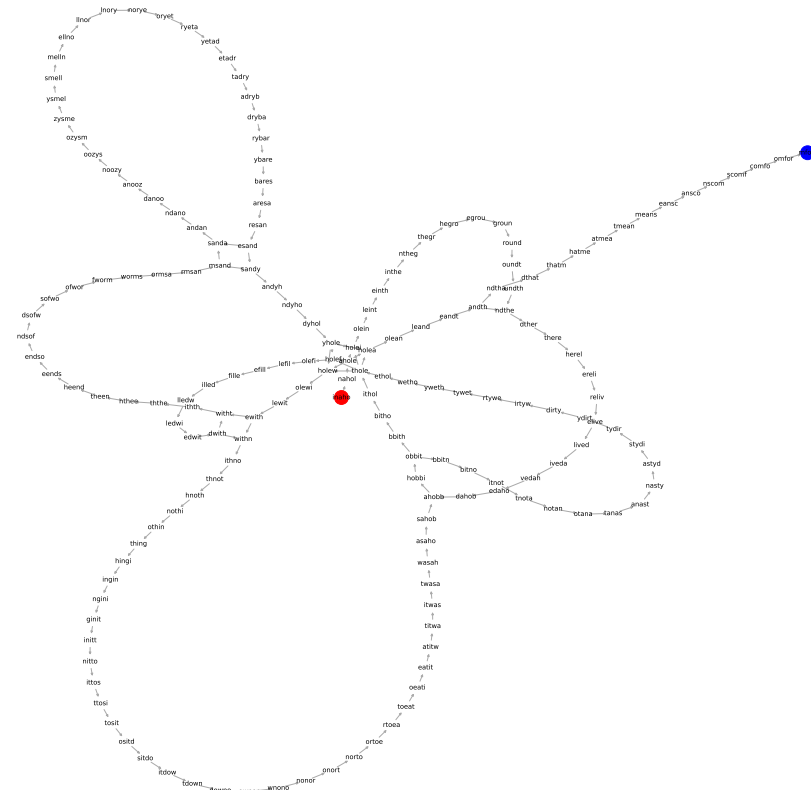
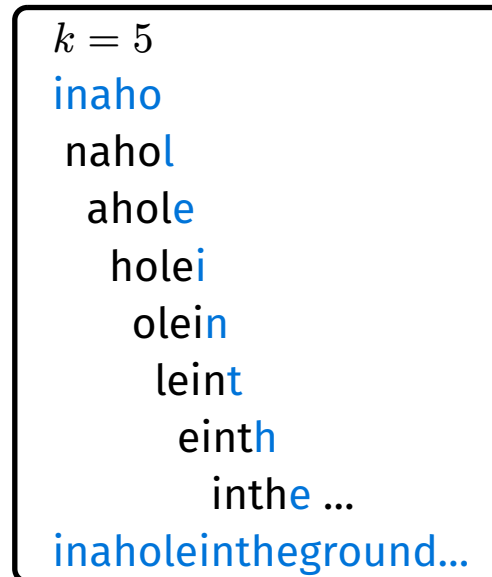


Figure 2: A graph of the 5-length kmers from the first sentence of *The Hobbit*.

This kind of graph is composed of kmers with $k-1$ overlapping characters.
AKA a De Bruijn graph:



$k = 5$
inaho
 nahol
 ahole
 holei
 olein
 leint
 einth
 inthe ...
inaholeintheground...

Following each edge along the graph, the original sentence can be constructed using the last letter of each word.

MY APPROACH

Instead of exploring the entire graph, anchor on the query and explore outward in both directions.

Genome:

in a hole in the **ground** there lived a hobbit

Query: **ground** → kmers: gro, rou, oun, und

Left (from “gro”): ...ein → int → nth → the → heg → egr → **gro**

Right (from “und”): **und** → ndt → dth → the → her → ere...

Result: ...ein the **ground** there...

- Identify kmers that make up the query (*core cloud*)
- Explore left from the leftmost query kmer (DFS)
- Explore right from the rightmost query kmer (DFS)
- Join the longest left + right contigs around the query

k → *kmer size*

Too small: same kmer appears everywhere (high coverage, low specificity)

Too large: kmers exceed read length (low coverage, high specificity)

Balance uniqueness vs. frequency

t → frequency threshold

Too low: sequencing errors enter the graph as noise

Too high: real but rare kmers are discarded

Balance signal vs. noise

max_visits → cycle limit

Prevents infinite loops through repeated sequence

Fixed at 2: allows passing through a node twice before stopping

Handle repeats without endless looping

RESULTS

Test 1

STRTABCDEFGGABCHIJKDEFENDSTRTABCDEFGGABCHIJKDEFEND

- Simple linear sequence with cycle at **DEF**
- Query at the edge of the sequence

Test 2

in a hole in the ground there lived a h**obbit** not a n**asty** dirty wethole...

- Central query
- Explores cycles caused by repeated words like “hobbit” (need for `max_visits` parameter)

Test 3

ATCTGCTGA...

- Simulated DNA reads (A, T, C, G)
- Tests performance with a realistic four-letter alphabet and larger read count

Unit Tests

- Test each function against expected output (integrated into GitHub actions)

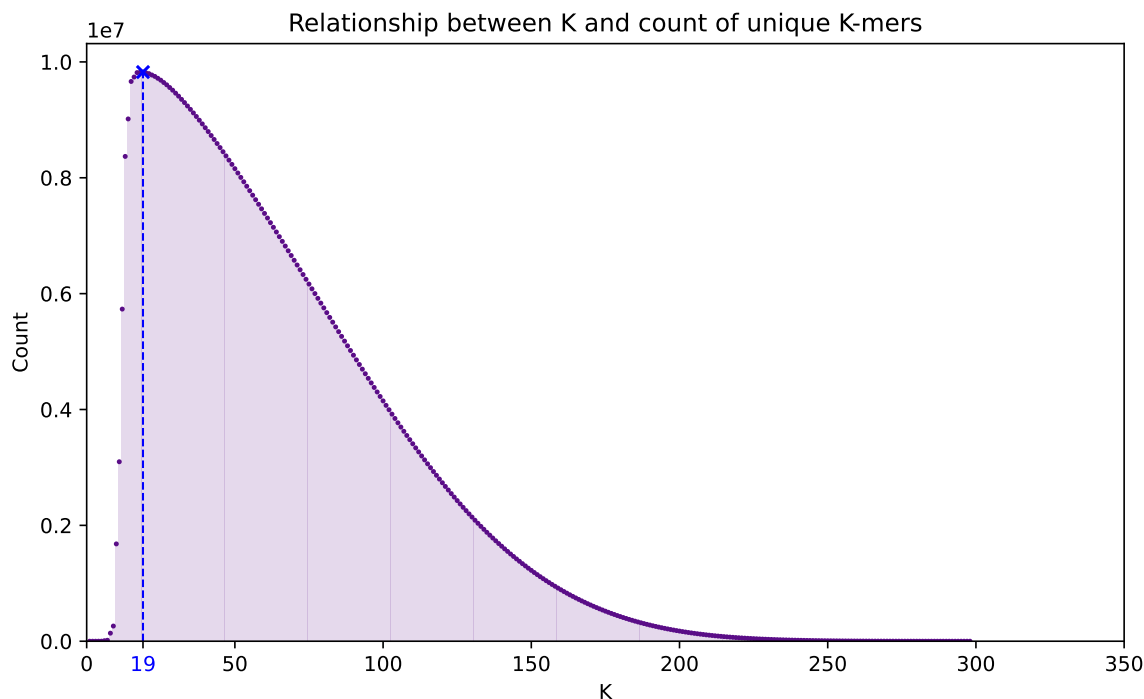


Figure 3: Unique kmers count vs. k

- **Small k ($< \sim 10$):** nearly every kmer seen repeatedly, very few unique
- **Medium k (~ 25):** unique count peaks; long enough that most kmers are seen only in one read, short enough to still be fully contained within a read
- **Large k ($\sim > 25$):** kmers approach read length; count collapses

Chosen value: **k = 19**

```
TTCTCCTCTTCCTCATCTCCGGCCTTTTCGACCTGCAGCCA
ATATGGGATCGGCCATTGAACAAGATGGATTGCACGCAGG
TTCTCCGGCCGCTTGGGTGGAGAGGCTATTCGGCTATGAC
TGGGCACAACAGACAATCGGCTGCTCTGATGCCGCCGTGT
TCCGGCTGTCAGCGCAGGGGCGCCCGGTTCTTTTTGTCAA
GACCGACCTGTCCGGTGCCCTGAATGAACTGCAGGACGAG
GCAGCGCGGCTATCGTGGCTGGCCACGACGGGCGTTCCTT
GCGCAGCTGTGCTCGACGTTGTCACTGAAGCGGGAAGGGA
CTGGCTGCTATTGGGCGAAGTGCCGGGGCAGGATCTCCTG
TCATCTCACCTTGCTCCTGCCGAGAAAGTATCCATCATGG
CTGATGCAATGCGGCGGCTGCATACGCTTGATCCGGCTAC
CTGCCCATTCGACCACCAAGCGAAACATCGCATCGAGCGA
GCACGTACTCGGATGGAAGCCGGTCTTGTCGATCAGGATG
ATCTGGACGAAGAGCATCAGGGGCTCGCGCCAGCCGAAC
GTTCCGCCAGGCTCAAGGCGCGCATGCCCGACGGCGATGAT
CTCGTCGTGACCCGATGGCGATGCCTGCTTGCCGAATATCA
TGGTGGAATGGCCGCTTTTCTGGATTACGACTGTGG
CCGGCTGGGTGTGGCGGACCGCTATCAGGACATAGCGTTG
GCTACCCGCTGATATTGCTGAAGAGCTTGGCGGCGAATGGG
CTGACCGCTTCCTCGTGCTTTACGGTATCGCCGCTCCCGA
TTCGCAGCGCATCGCCTTCTATCGCCTTCTTGACGAGTTC
TTCTGAGGGGATCCGCTGGGAGTTCAAAGGAGGGGTCCCC
TATCGAATTACAGTGACTGCAGTATATTCTGGAGGATTAG
CTGCTGCACCCTCAGTTTGGGA
```

BLASTn result:

Mus musculus Col1a2-G610C allele

(type I procollagen alpha2 chain, exon 33)

What this means:

Col1a2 encodes a structural protein in bone. The G610C mutation is a well-characterized mouse model of **osteogenesis imperfecta**, aka, brittle bone disease. This sequence likely derives from a transgenic mouse used to study that condition.

Query sequence shown in purple.

CONCLUSIONS

- Anchoring assembly on a known query makes targeted *de novo* assembly tractable
 - exploration starts at most relevant core
- Without ground truth, parameter tuning relies on test performance
- The assembled contig aligns confidently to the *Mus musculus* Col1a2-G610C transgenic allele which is consistent with a mouse model for osteogenesis imperfecta

Current limitations:

- No explicit error correction: the frequency threshold τ is a blunt proxy
- No ground truth so correctness cannot be confirmed

Future directions:

- Coverage-weighted edges
 - trust high-coverage paths more than low-coverage ones
- Bubble detection to replace reliance on a fixed `max_visits` parameter

REFERENCES

[1] J. R. R. Tolkien, *The Hobbit*. HarperCollins, 2012.

<https://github.com/canderson318/research-methodology-programming-assignment>